

Backend Architecture — nativetalk / UCP Platform

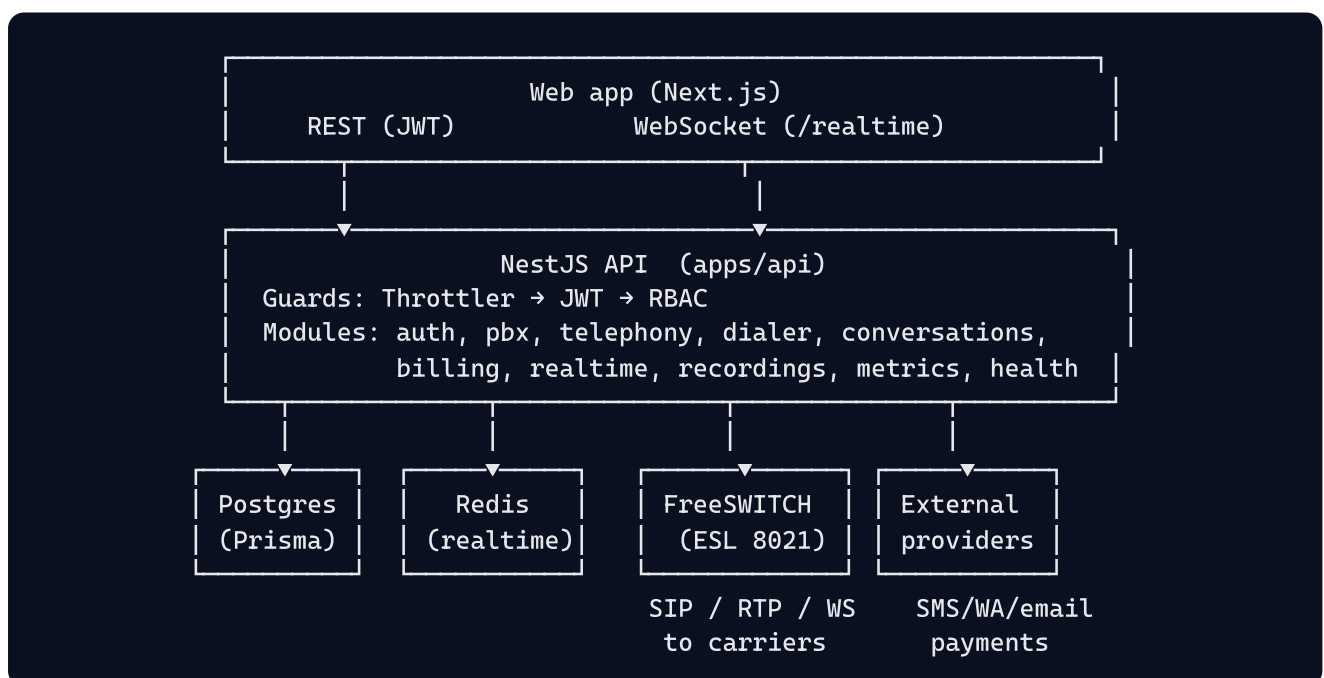
Audience: an engineer joining the project who needs to understand the backend end-to-end. **Scope:** the `apps/api` service — a NestJS + Prisma + PostgreSQL backend that drives a multi-tenant Cloud PBX, Contact Center, and Omnichannel Inbox on top of FreeSWITCH. **Status:** Phase 6 (hardening / go-live). Generated 2026-06-24.

1. What this system is

The platform ("UCP" — Unified Communications Platform, branded **nativetalk**) is a **multi-tenant SaaS** that gives each customer company its own:

- **Cloud PBX** — SIP extensions, trunks, ring groups, IVRs, ACD queues, inbound routing, time conditions.
- **Contact Center** — outbound dialer (progressive / power / voice-broadcast), dispositions, DNC, lead groups, call recording, reports.
- **Omnichannel Inbox** — unified conversations across voice, SMS, WhatsApp, email, and webchat.
- **Browser softphone** — agents take and make calls in the browser over WebRTC.
- **Billing / plans** — per-tenant subscription plans, usage limits, invoices.

The call-processing engine is **FreeSWITCH**. The NestJS API is the *control plane*: it owns the database, renders FreeSWITCH configuration from that database, drives calls via the Event Socket, and exposes a REST + WebSocket API to the Next.js web app.



2. Technology stack

Layer	Choice	Notes
Runtime	Node.js ≥ 20	monorepo via npm workspaces (<code>apps/*</code> , <code>packages/*</code>)
Framework	NestJS 11	modular DI, decorators, global guards
ORM	Prisma 6 on PostgreSQL	single schema, <code>tenantId</code> on every business table
Auth	<code>@nestjs/jwt</code> + <code>passport-jwt</code> + <code>bcryptjs</code>	stateless JWT bearer tokens
Telephony	FreeSWITCH via <code>esl</code> (Event Socket Library)	one persistent, auto-reconnecting socket
Realtime	<code>socket.io</code> via <code>@nestjs/websockets</code>	<code>/realtime</code> namespace, 2s snapshots
Cache / state	Redis via <code>ioredis</code>	realtime state, future BullMQ queue
Rate limiting	<code>@nestjs/throttler</code>	global 120 req/min/IP; login & signup stricter
Security headers	<code>helmet</code>	applied globally
Metrics	<code>prom-client</code>	Prometheus <code>/metrics</code> endpoint
Validation	<code>class-validator</code> + <code>class-transformer</code>	global <code>ValidationPipe</code> (whitelist + transform)

The API package is `@ucp/api`. Build with `nest build`; run `node dist/main.js`. Dev: `npm run api:dev`.

3. Application bootstrap & the request pipeline

Bootstrap — `src/main.ts`

1. **Fail-fast in production** if `JWT_SECRET` is unset or still the dev default.
2. `helmet()` for security headers.
3. **CORS** restricted to `WEB_ORIGIN` (comma-separated origins; default `http://localhost:3001`).
4. Global `ValidationPipe({ whitelist: true, transform: true })` — strips unknown body fields, coerces types.
5. Global `AllExceptionsFilter` — uniform JSON errors.
6. Listens on `PORT` (default **4000**), bound to `0.0.0.0`.

Module graph — `src/app.module.ts`

`ConfigModule` is global. `ThrottlerModule` sets the global rate limit. Then every feature module is imported. Three **global guards run on every route, in order**:

```
{ APP_GUARD: ThrottlerGuard } // 1. rate limit
{ APP_GUARD: JwtAuthGuard }   // 2. authenticate (unless @Public)
{ APP_GUARD: RbacGuard }      // 3. authorize (role / permission)
```

Order matters: a request is rate-limited first, then authenticated, then authorized. `@Public()` routes skip auth + RBAC.

Error handling — `src/common/all-exceptions.filter.ts`

Catches everything. Known `HttpException`s keep their status/message; anything else becomes a generic **500** in production (no stack leakage) but is always logged. Response shape:

```
{ "statusCode": 403, "message": "...", "path": "/pbx/extensions", "timestamp": "... " }
```

4. Authentication & authorization

This is the heart of the security model — read this section carefully.

4.1 Login & tokens — `src/auth/`

- `POST /auth/login` (`@Public` , throttled 8/min/IP) → `AuthService.login()` .
- Looks up the **Manager** by email **globally** (across all tenants), `bcrypt.compare` s the password, stamps `lastLoginAt` , writes an `AuditLog` , and signs a JWT.
- **JWT payload** is minimal: `{ sub: managerId, tenantId, email }` . Secret = `JWT_SECRET` , expiry = `JWT_EXPIRES_IN` (default **12h**).
- `POST /auth/logout` is a no-op (token is stateless; client discards it). A Redis revocation list is the noted future hook.
- `GET /auth/me` returns the current resolved user.

⚠ **Important consequence:** because login resolves accounts by email across *all* tenants, **email must be globally unique among Managers**. The self-service signup flow enforces this explicitly (see §11).

4.2 Per-request identity — `src/auth/jwt.strategy.ts`

On **every** authenticated request the `JwtStrategy.validate()` re-loads the Manager + its role from the DB (not from the token). This means **permission and role changes take effect immediately** — nothing stale is baked into the token. It rejects disabled/missing accounts. The resolved `AuthUser` is attached to `req.user` :

```
interface AuthUser {
  id; tenantId; email;
  role; // 'admin' | 'manager' | 'agent'
  adminAccess; // from the manager's role (full company access)
  superAdmin; // cross-tenant platform operator (Tech4mation)
  permissions; // { section: { enabled, items: { key: bool } } }
}
```

4.3 The three access tiers — `src/common/rbac.guard.ts`

`RbacGuard` runs globally after JWT. Logic:

1. `@Public()` → allow.
2. No `req.user` → `403`.
3. **Full-access tiers** → allow everything in their scope: `superAdmin`, `role === 'admin'`, or `adminAccess`.
4. **Agents** (`role === 'agent'`) → **denied by default**; allowed only on routes explicitly tagged `@Roles('agent')` (e.g. softphone, contacts, set-own-status).
5. **Managers** → allowed if the route names them in `@Roles(...)`, **or** if they hold the route's `@Permissions(...)`. Routes with no `@Permissions` are allowed.

Permission strings are `"section"` or `"section:item"`, checked against `permissions[section].enabled` / `permissions[section].items[item]`.

4.4 Decorators — `src/common/decorators.ts`

Decorator	Effect
<code>@Public()</code>	skip JWT + RBAC
<code>@Permissions('pbx')</code> , <code>@Permissions('users:user')</code>	manager must hold this permission
<code>@Roles('agent')</code>	allow this role on the route (used to grant agents specific access)
<code>@CurrentUser()</code>	inject the <code>AuthUser</code> (or a field of it)

4.5 Super-admin — `src/common/super-admin.guard.ts`

`SuperAdminGuard` allows **only** `Manager.superAdmin` users. It guards the **platform** (cross-tenant) endpoints — tenant onboarding, plan management. A clever idiom used elsewhere:

`@Permissions('admin')` is a permission **nobody is ever granted**, so manager-role and manager administration effectively require `adminAccess` (only admins pass the full-access shortcut).

5. Multi-tenancy model

- **Every business table carries** `tenantId` (see `schema.prisma`). A single deployment serves many isolated companies.
- Isolation is enforced **in application code**, not via Postgres RLS: every query filters by `u.tenantId`, which comes from the verified JWT → DB-loaded Manager. There is no way for a client to set their own `tenantId` (the generic CRUD helper strips it from request bodies).
- The generic tenant-scoped CRUD helper — `src/resources/crud.ts` — is the workhorse:

```
list:    findMany({ where: { tenantId } })
create: create({ data: { ...sanitize(body), tenantId } })
update: updateMany({ where: { id, tenantId }, data: sanitize(body) })
remove: deleteMany({ where: { id, tenantId } })
```

`sanitize()` strips `id`, `tenantId`, `createdAt`, `updatedAt` so a client can never set them. `update` / `remove` use `updateMany` / `deleteMany` with the `tenantId` in the WHERE clause — so a cross-tenant id silently affects **zero** rows rather than leaking.

6. Module-by-module reference

6.1 Infrastructure modules

Module	Provides
<code>PrismaModule</code> / <code>PrismaService</code>	<code>PrismaClient</code> with connect/disconnect lifecycle hooks.
<code>RedisModule</code> / <code>RedisService</code>	lazy-connecting <code>ioredis</code> client; degrades gracefully if Redis is down; <code>ping()</code> for health.
<code>FreeswitchModule</code> (<code>@Global</code>)	exports <code>FreeswitchService</code> + <code>FsProvisioningService</code> to the whole app.

6.2 Auth — `src/auth/`

Login, JWT strategy, `AuthService.hash()` (bcrypt). Exported so other modules (accounts, managers, billing) can hash passwords.

6.3 Identity & RBAC administration

Controller	Route	Guard	Purpose
<code>ManagersController</code>	<code>/managers</code>	<code>@Permissions('admin')</code> → admins only	CRUD login accounts (managers); never leaks <code>passwordHash</code> .
<code>ManagerRolesController</code>	<code>/manager-roles</code>	<code>@Permissions('admin')</code> → admins only	CRUD RBAC roles (named permission sets).
<code>AccountsController</code>	<code>/accounts</code>	<code>@Permissions('users')</code>	In-company account management: create agents/managers/admins. Only admins may create admins. Super-admin accounts are hidden from non-super-admins.
<code>ResourcesModule</code>	<code>/users</code> , <code>/user-roles</code> , <code>/user-groups</code> , <code>/outgoing-rules</code>	per-section <code>@Permissions</code>	Four near-identical tenant-scoped CRUD resources (desk users / groups / roles / dialing rules) built on the generic <code>crud()</code> helper.

Manager vs User: a **Manager** is a *login account* (admin/manager/agent). A **User** is a *desk/agent record* mapping to a SIP device — not yet a login. They are deliberately separate models.

6.4 Generic collections — `src/collections/`

`/collections/:collection` — a JSON-backed store (`DataRecord`) for contact-center collections whose relational models arrive later. Whitelisted names only (`outbound-campaigns` , `dispositions` , `dnc` , `webforms` , `lead-groups` , ...), tenant-scoped, guarded by `@Permissions('contact-center')` . Rows are stored as JSON and flattened on read.

6.5 FreeSWITCH integration — `src/freeswitch/`

This is the most important subsystem. Two services:

FreeswitchService — owns **one persistent, auto-reconnecting Event Socket** (ESL) connection to FreeSWITCH (default `127.0.0.1:8021` , password `FS_PASSWORD`). Unlike the legacy app (one socket per call) this is long-lived — the foundation for realtime events and dialer scale.

- On connect it **ensures required modules** are loaded (`mod_callcenter` for ACD, `mod_avmd` for answering-machine detection) — they aren't always in the default autoload set.
- `api(cmd)` — blocking command; throws a clear error if FS is down (no hangs).
- `bgapi(cmd)` — non-blocking; returns the Job-UUID.
- `status()` — health probe.

FsProvisioningService — turns the **database PBX config into FreeSWITCH XML** and applies it. This is the DB→telephony bridge:

- Pure generators produce XML per category: extensions → `directory/default/zzz_ucp_users.xml`, trunks → `sip_profiles/external/zzz_ucp_gw.xml`, dialplan (ring groups, inbound routes, IVRs, queues, time conditions) → `dialplan/default/zzz_ucp.xml`, ACD queues → `autoload_configs/callcenter_ucp_queues.xml`.
- **One file per category, overwrite = implicit cleanup** of deleted entities.
- `apply()` writes the files, wires `mod_callcenter` to include our queues, then `reloadxml` + rescans the external SIP profile + live-provisions callcenter agents/tiers.
- **Graceful degradation:** if the conf dir isn't writable (e.g. Program Files without admin) it returns the generated XML instead of throwing, so it can be applied on a writable host.
- All values are XML-escaped (`esc()`); IVRs are implemented with `play_and_get_digits` so they work without `mod_ivr`.

6.6 PBX — `src/pbx/`

- `PbxService.loadConfig(tenantId)` loads all seven PBX entity types; `sync()` applies them; `preview()` generates XML without writing; `safeSync()` is fire-and-forget (never throws into a request).
- **PbxModule controllers** are produced by a factory `pbxController(path, model)` — tenant-scoped CRUD that **re-syncs FreeSWITCH after every mutation**, so the live phone system always matches the DB. Routes (all `@Permissions('pbx')`): `/pbx/extensions`, `/pbx/trunks`, `/pbx/ring-groups`, `/pbx/inbound-routes`, `/pbx/ivr`s, `/pbx/queues`, `/pbx/time-conditions`.
- `POST /pbx/sync` and `GET /pbx/preview` apply/preview the whole tenant config.

6.7 Telephony — `src/telephony/`

Agent/supervisor live call features (`TelephonyService`):

- `GET /telephony/softphone?extension=` (`@Roles('agent')`) — everything a browser softphone needs to register: SIP secret, domain, WS URL (`FS_WS_URL`, default `ws://domain:5066`), STUN server. Tenant-scoped + auth-protected.
- `GET /telephony/calls` — live channels (`show calls as json`).
- `POST /telephony/calls/:uuid/monitor` — supervisor **listen / whisper / barge** via `eavesdrop` (agents denied by RBAC).
- `GET /telephony/agents` — live ACD roster.
- `POST /telephony/agents/:extension/status` (`@Roles('agent')`) — agent sets own `mod_callcenter` status (`Available`, `On Break`, ...).

6.8 Dialer — `src/dialer/`

The outbound campaign engine (`DialerService`). Runs are **in-memory** (`Map<campaignId, Run>`); each attempt is **persisted to Postgres** (`CallAttempt`) for reporting.

- **Two modes:** `agent` (answered calls bridged into an ACD queue → connected to a live agent: progressive/power dialing) vs `broadcast` (play an audio file). Determined by whether the campaign has a `queue`.
- `start()`: gathers numbers from the paste field + attached lead group, de-dupes, marks DNC numbers as `skipped` up front, then runs the dial loop.
- `loop()`: concurrency = 1 for manual/preview, or campaign `concurrency` (clamped 1–10) for progressive/power — then **further clamped by the tenant's plan** `maxConcurrentCalls` limit. Retries retryable causes (NO_ANSWER, USER_BUSY, ...) up to `maxAttempts`.
- `dialOne()`: originates via `sofia/gateway/<gw>/<number>` when a gateway is set, else `user/<number>` (internal only). Sets caller-ID, optional `record_session`, then bridges to the queue or plays audio.
- Endpoints (`@Permissions('contact-center')`): campaigns/dispositions/dnc/lead-groups/leads CRUD; `POST /campaigns/:id/start|stop`, `GET /campaigns/:id/run`, `POST /campaigns/:id/disposition`.
- **Reports** (`@Permissions('reports')`): `/reports/campaign` (aggregates), `/reports/cdr` (recent attempts), `/reports/agent-performance` (live from mod_callcenter).
- `DialerService` is **exported** and reused by the realtime gateway.

6.9 Recordings — `src/recordings/`

Streams WAV recordings straight from the FreeSWITCH recordings folder (`FS_RECORDINGS_DIR`).

`@Permissions('reports')`. Supports HTTP **Range** requests (seeking). **Path-traversal-safe**: only the stored `basename` is used and the resolved path must stay inside `REC_DIR`.

6.10 Realtime — `src/realtime/`

A `socket.io` gateway on the `/realtime` namespace.

- **JWT-authenticated handshake**: token from `handshake.auth.token`; invalid → disconnect.
- Builds a **live snapshot** from FreeSWITCH (`callcenter_config agent list`, `queue list`, `show calls as json`) + active dialer runs, and **emits it to all clients every 2 seconds** — but only while clients are connected (the timer stops at zero clients).
- Snapshot includes agents, queues, active calls, campaign runs, and a summary (available / on-call / on-break counts).

6.11 Conversations / Omnichannel — `src/conversations/`

Unified inbox across `voice | sms | whatsapp | email | webchat`.

- `ConversationsController` (`@Permissions('omnichannel')`): list/get conversations, create, send message or internal note, assign, set status.
- `ChannelAdapter` **seam** (`channel-adapter.ts`): each channel implements `send()`. `webchat / voice` are handled in-app; `sms / whatsapp / email` are `UnconfiguredAdapter`s that *queue* outbound messages (never lose them) until a real provider (Termii/Twilio, Meta WhatsApp, SMTP) is wired in — **nothing else in the app changes** when you swap one in.
- **Inbound webhook** `POST /channels/:channel/inbound` (`@Public`) — how providers deliver inbound messages (and how inbound is simulated in dev). Upserts the contact, reuses the latest

open conversation or opens one.

- `upsertContact()` de-dupes contacts by phone/email.
- Canned responses CRUD under `/canned-responses`.

6.12 Contacts — `src/contacts/`

Company contacts (customers). `@Roles('agent')` — a core agent capability, also open to managers/admins. Tenant-scoped search by name/phone/email.

6.13 Billing / SaaS — `src/billing/`

Three controllers + a service:

- `PlatformController` `/platform/*` — `@UseGuards(SuperAdminGuard)`, **super-admin only**: list tenants, **onboard tenant**, suspend/activate, set plan, usage, generate invoice, mark paid; plan CRUD.
- `BillingController` `/billing/*` — any authenticated manager of the tenant: `me` (plan/subscription/usage/limits), invoices, pay invoice, set branding.
- `BrandingController` `/branding` (`@Public`) — so the login page can theme before auth.
- `SignupController` `/signup` (`@Public`, throttled) — self-service registration (see §11).
- `effectiveLimits()` layers limits: `DEFAULT_LIMITS` ← plan limits ← tenant overrides. Used by the dialer to cap concurrency.
- **Payment seam** (`payment.ts`): same pattern as channels — an `UnconfiguredProvider` until Paystack/Flutterwave keys are supplied; until then checkout is "manual" (invoice stays open, can be marked paid by hand).

6.14 Observability — `src/metrics/`, `src/health/`

- `/metrics` (`@Public`) — Prometheus exposition: Node/process defaults + HTTP request counter & duration histogram. A middleware labels by **route pattern** (not raw URL) to keep cardinality low.
- `/health` (`@Public`) — checks DB (hard dependency), Redis, FreeSWITCH. `/health/live` (liveness, no deps) and `/health/ready` (readiness, DB must be reachable) for orchestrators.

7. Data model (Prisma / PostgreSQL)

Every business model has `id` (cuid), `tenantId`, timestamps, and a `@@index([tenantId])`. Grouped by domain:

Identity & tenancy `Tenant` (slug, status `trial|active|suspended`, plan, limits, branding) · `Manager` (login account; role admin/manager/agent; `superAdmin`; `agentExtension`) · `ManagerRole` (RBAC permissions JSON) · `User` / `UserRole` / `UserGroup` (desk users) · `OutgoingRule` · `DataRecord` (JSON collections) · `AuditLog`.

Cloud PBX `Extension` (SIP device: password, context, callerId, `tollAllow`) · `Trunk` (Sofia gateway to a carrier) · `RingGroup` · `InboundRoute` (DID → destination) · `Ivr` · `Queue` (ACD) · `TimeCondition`.

Contact Center `Disposition` · `Dnc` · `LeadGroup` / `Lead` · `OutboundCampaign` (dialMethod, gateway, queue, callerId, concurrency, recording, amd) · `CallAttempt` (CDR row per attempt) · `Callback` .

Omnichannel `Contact` · `Conversation` (channel, status, assignee, lastMessageAt) · `Message` (direction, type message/note, externalId, status) · `CannedResponse` .

Billing `Plan` (priceMonthly in minor units, currency, limits, features) · `Subscription` · `Invoice` (lineItems, provider).

Cascade rules: deleting a `Tenant` cascades to all its data; nullable FKs (plan, lead group, campaign on attempt) use `SetNull` so history survives.

8. Telephony data flow (DB → FreeSWITCH → call)

This ties the pieces together — the single most important flow to understand.

Provisioning (config) flow:

```
Admin edits an Extension/Trunk/Queue via /pbx/*
→ crud() writes the row (tenant-scoped)
→ PbxService.safeSync(tenantId)
→ loadConfig() reads all PBX rows
→ FsProvisioningService.generateAll() → XML files
→ write to FreeSWITCH conf dirs (overwrite)
→ fs.api('reloadxml') + 'sofia profile external rescan'
→ live mod_callcenter agent/tier provisioning
```

The DB is the **source of truth**; FreeSWITCH config is **regenerated**, never hand-edited.

Outbound call flow (dialer, agent mode):

```
POST /campaigns/:id/start
→ DialerService gathers numbers, removes DNC
→ for each (respecting concurrency ≤ plan limit):
    originate {caller-id,record}sofia/gateway/<trunk>/<number> &callcenter(<queue>@c
→ on answer: caller enters the ACD queue → a live agent is bridged
→ every attempt persisted as a CallAttempt (status, cause, disposition, recording)
```

Inbound call flow: a DID hits FreeSWITCH → matches a generated `InboundRoute` dialplan extension → routes to an extension / ring group / IVR / queue / voicemail / hangup.

Agent media: the browser softphone registers over **WebSocket (WS)** + **WebRTC** to FreeSWITCH using the config from `/telephony/softphone` ; STUN handles NAT. Supervisors monitor via `eavesdrop` (listen/whisper/barge).

The trunk is the gate for external calls. With no `Trunk` , the dialer falls back to `user/<number>` (internal extensions only). A `Trunk` row becomes a Sofia gateway in `sip_profiles/external` . This is exactly the seam where carrier (VoIPSwitch) provisioning plugs in.

9. External integration seams

The backend is built around **pluggable seams** so unconnected providers degrade gracefully instead of failing:

Seam	File	Default (unconfigured)	Plug in
Messaging channels	<code>conversations/channel-adapter.ts</code>	queues outbound msgs	Termii/Twilio (SMS), Meta (WhatsApp), SMTP (email)
Payments	<code>billing/payment.ts</code>	"manual" checkout (mark paid by hand)	<code>PAYSTACK_SECRET_KEY</code> / <code>FLUTTERWAVE_SECRET_KEY</code>
Carrier trunk	<code>Trunk</code> model + <code>FsProvisioningService</code>	internal calls only	a SIP gateway row (e.g. VoIPSwitch / Nativetalk)

The pattern is identical everywhere: an interface, an `Unconfigured*` stub, and a registry/factory that swaps in the real implementation from env — **no other code changes**.

10. Configuration (environment variables)

Variable	Default	Used by
<code>DATABASE_URL</code>	— (required)	Prisma / Postgres
<code>REDIS_URL</code>	<code>redis://localhost:6379</code>	RedisService
<code>JWT_SECRET</code>	<code>dev-secret</code> (rejected in prod)	auth, realtime
<code>JWT_EXPIRES_IN</code>	<code>12h</code>	token expiry
<code>WEB_ORIGIN</code>	<code>http://localhost:3001</code>	CORS allow-list (comma-sep)
<code>PORT</code>	<code>4000</code>	HTTP listener
<code>NODE_ENV</code>	—	prod guards + error masking
<code>FS_HOST</code> / <code>FS_PORT</code> / <code>FS_PASSWORD</code>	<code>127.0.0.1</code> / <code>8021</code> / <code>ClueCon</code>	Event Socket
<code>FS_REQUIRED_MODULES</code>	<code>mod_callcenter, mod_avmd</code>	auto-loaded on connect
<code>FS_CONF_DIR</code>	<code>C:/Program Files/FreeSWITCH/conf</code>	where generated XML is written
<code>FS_WS_URL</code>	<code>ws://<domain>:5066</code>	browser softphone signalling
<code>FS_STUN_URL</code>	<code>stun:stun.l.google.com:19302</code>	WebRTC NAT traversal
<code>FS_RECORDINGS_DIR</code>	<code>C:/Program Files/FreeSWITCH/recordings</code>	recording playback
<code>PAYMENT_PROVIDER</code>	<code>paystack</code>	payment seam selection

11. Self-service signup (added in this phase)

A public registration flow now sits alongside the operator-driven onboarding:

- `GET /signup/plans` (`@Public`) — active plans for the signup page.
- `POST /signup` (`@Public` , throttled **5/min/IP**) → `BillingService.registerTenant()` :
 - validates company/email/password (min 8 chars, email format);
 - **global** email-uniqueness check (because login is by email across tenants);
 - creates a tenant in `trial` status + an "Administrator" role + the first admin Manager;
 - optional plan, validated against active plans.
- The web client then logs in with the same credentials.

This contrasts with `POST /platform/tenants (onboardTenant)`, which is **super-admin only** and creates tenants in `active` status. The two share the same tenant-bootstrap shape but differ in who may call them and the starting status.

12. Security posture (summary)

- **Stateless JWT**, re-validated against the DB every request (instant revocation of disabled accounts / permission changes).
- **Three global guards**: throttle → authenticate → authorize. Default-deny for agents.
- **Tenant isolation** enforced in every query via `tenantId` from the verified token; bodies can't set `tenantId / id`.
- **Password hashing** with bcrypt; hashes never serialized to responses.
- **Helmet** headers, **CORS allow-list**, **rate limiting** (global + stricter on login/signup).
- **Prod hardening**: refuses default JWT secret; masks internal errors; logs all 5xx.
- **Path-traversal protection** on recording streaming.
- **Audit log** on login (extensible to other actions).

Known gaps / future hooks (called out in code): no JWT revocation list yet (logout is client-side); tenant isolation is app-enforced not RLS; messaging/payment providers are stubs; recordings are local-disk (no object storage); dialer run state is in-memory (lost on restart — only persisted `CallAttempt`s survive).

13. How to run & build

```
# infra (Postgres + Redis)
npm run infra:up

# API
npm --workspace @ucp/api run prisma:deploy # apply migrations
npm run api:dev                          # watch mode (port 4000)
npm run api:build                         # production build → dist/

# health check
curl http://localhost:4000/health
```

FreeSWITCH must be reachable on the ESL port for telephony features; the API still boots and serves non-telephony routes if it isn't (commands return clear errors).

Tech4mation — nativetalk / UCP platform. Backend architecture reference.
